OS Core Simulator - Complete Project Documentation

Vinay Sharma
Department of Computer Science
Chandigarh College of Engineering and Technology
lco24382@ccet.ac.in

Abstract

This project presents a comprehensive operating system simulation platform that models the core functionalities of an OS, including process management, memory management, disk scheduling, file systems, deadlock handling, and inter-process communication. The system is developed using Python with a graphical interface to visualize operations in real time. The simulator allows users to understand and analyze different OS algorithms and concepts interactively. Experimental observations show how various algorithms affect performance metrics such as waiting time, memory utilization, and disk efficiency.

1 Introduction

Operating systems are fundamental components of modern computing systems, responsible for managing hardware resources and providing services to applications. Understanding OS concepts such as scheduling, memory allocation, and synchronization is essential but often difficult due to their abstract nature.

Traditional teaching methods rely heavily on theory, making it challenging for students to visualize real-time behavior. To address this limitation, simulation-based learning provides an effective solution.

In this project, we developed an **OS Core Simulator**, which integrates multiple OS components into a single platform. The simulator provides real-time visualization and interaction with various subsystems, helping users better understand how operating systems function internally.

2 Problem Statement

The objective of this project is to:

- Simulate core operating system functionalities
- Provide visualization of OS algorithms
- Compare performance of different scheduling and memory techniques
- Help users understand OS concepts practically

3 System Design

The system is designed as a modular architecture consisting of multiple subsystems:

- **Process Management Module**
- **Memory Management Module**
- **Disk & File System Module**
- **Scheduling Module**
- **Deadlock Handling Module**
- **Inter-Process Communication Module**
- **User Interface Module**

Each module operates independently but interacts with others to simulate a complete OS environment.

4 Methodology

The system follows a structured approach for simulation:

4.1 Process Management

Processes are created with attributes such as arrival time, burst time, and priority. Different scheduling algorithms are implemented:

- FCFS (First Come First Serve)
- SJF (Shortest Job First)
- Round Robin
- Priority Scheduling

The scheduler selects processes based on the chosen algorithm and calculates waiting time and turnaround time.

4.2 Memory Management

The simulator implements paging and page replacement techniques.

- FIFO (First In First Out)
- LRU (Least Recently Used)
- Optimal Algorithm

Memory is divided into frames, and pages are allocated dynamically. Page faults are tracked and analyzed.

4.3 Disk Scheduling

Disk requests are processed using different algorithms:

- FCFS
- SSTF (Shortest Seek Time First)
- SCAN
- C-SCAN

The simulator calculates total seek time and visualizes head movement.

4.4 Deadlock Handling

Deadlock detection and prevention are implemented using:

- Resource Allocation Graph
- Banker's Algorithm

The system checks safe states and identifies potential deadlocks.

4.5 File System Simulation

File allocation methods implemented:

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

The system manages file creation, deletion, and storage allocation.

4.6 Inter-Process Communication

IPC mechanisms include:

- Shared Memory
- Message Passing
- Pipes

These mechanisms allow processes to communicate efficiently.

5 Evaluation Metrics

The performance of the simulator is evaluated using:

- **Waiting Time**: Time a process waits before execution
- **Turnaround Time**: Total time taken by a process
- **Seek Time**: Disk head movement efficiency
- **Page Fault Rate**: Memory efficiency
- **CPU Utilization**: Processor efficiency

6 Results

The simulator demonstrates clear differences between algorithms:

- **FCFS**: Simple but leads to high waiting time
- **SJF**: Reduces waiting time but requires prior knowledge
- **Round Robin**: Provides fairness but increases context switching
- **LRU vs FIFO**: LRU performs better in reducing page faults
- **SSTF**: Minimizes seek time but may cause starvation

The results show that no single algorithm is optimal for all scenarios, and trade-offs are necessary.

7 Discussion

The OS Core Simulator provides an interactive and practical environment to study complex operating system concepts that are otherwise difficult to understand through theory alone. Unlike traditional classroom learning, this system allows users to actively experiment with different algorithms and observe their behavior in real time. This hands-on approach significantly improves conceptual clarity and helps users understand how internal OS components function together.

One of the major strengths of the system is its **modular architecture**. Each component such as process management, memory management, disk scheduling, and deadlock handling is designed as an independent module. This makes the system highly flexible, allowing future enhancements, debugging, and feature additions without affecting the entire system.

Another important advantage is the **visualization capability**. The simulator provides graphical representations of processes, memory allocation, and disk operations. These visual insights make it easier to analyze algorithm performance, compare results, and understand concepts like fragmentation, page faults, and CPU scheduling.

However, the simulator operates in a **controlled and simplified environment**. It does not fully replicate real-world operating systems, which involve complex hardware interactions, multi-core processing, concurrency issues, and real-time constraints. Despite these limitations, the simulator effectively demonstrates the core principles of operating systems and serves as a strong educational tool.

8 Limitations

- **Simplified Simulation Environment**
  The system is designed for educational purposes and does not capture all complexities of a real operating system.
- **Limited Hardware-Level Accuracy**
  Hardware-level interactions such as CPU architecture, cache memory behavior, and I/O operations are not fully modeled.
- **Lack of Multi-Core Support**
  The current implementation mainly focuses on single or limited CPU simulation and does not fully represent multi-core or parallel processing environments.
- **Use of Assumed Parameters**
  Some parameters such as execution time, memory usage, and process arrival are simulated rather than collected from real systems, which may affect accuracy.
- **Scalability Constraints**
  The simulator may not perform efficiently when handling very large datasets or a high number of processes simultaneously.

9 Conclusion

This project successfully demonstrates the working principles of core operating system components through an integrated simulation approach. By combining multiple subsystems such as process scheduling, memory management, disk operations, and inter-process communication, the OS Core Simulator provides a comprehensive platform for understanding OS behavior.

The implementation highlights how different algorithms impact system performance metrics such as waiting time, turnaround time, and resource utilization. It also emphasizes the importance of selecting appropriate algorithms based on system requirements, as each approach involves certain trade-offs.

Overall, the project serves as an effective educational tool that bridges the gap between theoretical knowledge and practical understanding. It enables users to visualize, experiment, and analyze operating system concepts in an interactive manner.

10 Future Work

- **Multi-Core CPU Simulation**
  Extend the system to support multi-core processors and parallel execution to better reflect modern computing systems.
- **Integration with Real-Time Data**
  Incorporate real system data for parameters such as CPU usage and memory consumption to improve simulation accuracy.
- **Enhanced Graphical Visualization**
  Improve the user interface with advanced graphs, animations, and dashboards for better analysis and user experience.
- **Advanced Scheduling Algorithms**
  Implement more sophisticated algorithms such as Multi-Level Feedback Queue (MLFQ) and machine learning-based schedulers.
- **Cloud and Distributed System Simulation**
  Expand the project to simulate cloud environments and distributed systems for a broader understanding of modern computing architectures.